FINAL YEAR PROJECT

[SMART JOURNEY PLANNER APP]

[SUPERVISOR: DR. RABEEH AYAZ ABBASI]

[MUHAMMAD AHMAD]
[04072013010]
[2020-2024]

Acknowledgement

Assalam o Alikum,

Firstly, I would extend my gratitude to Allah Almighty from the bottom of my heart for guiding me during my time at Quaid-e-Azam University. Much appreciation is owed to my mentor, Dr Rabeeh Ayaz Abbasi, for his steadfast assistance and priceless suggestions. Special gratitude is also extended to Dr. Onaiza Maqbool, Dr. Ayyaz Hussain, Dr. Akmal Saeed Khattak, Dr. Khalid Saleem, Dr. Ghazanfar Farooq Siddique, Dr. Muhammad Shuaib Karim, Dr. Muazzam Ali Khan Khattak, Dr. Mudassar Azam Sindhu, Dr. Umer Rashid, Ms. Ifrah Farukh Khan, and Ms. Memoona Afsheen for their invaluable assistance throughout my journey in CS department. I would also like to convey my sincere appreciation to Sir Irfan ul Haq for his consistent support during the challenges encountered in this project. His support and valuable advice helped me gain crucial success during this project. Throughout this wonderful journey, I have not only gained industrial knowledge but have also applied my skills, intellect, and proficiency to the best of my ability. I am wholeheartedly grateful for the everlasting support of my family and friends especially Muhammad Bilal who has helped me achieve this milestone.

Abstract

People encounter numerous challenges when using public transportation due to a lack of detailed information about routes, schedules, fares, and arrival times etc. The goal of this project is to provide a user-friendly mobile application system for availing public transport services. Users will input their origin and destination, and the system, utilizing public transport's schedule and route information, will generate and display optimal travel routes on the display from google maps. The user will have the choice to pick route he deems suitable. These routes will be filtered based on several factors such as minimum vehicle changes, fastest routes, minimum fares etc. The system will feature an easy-to use interface, maintaining simplicity and effortless management. The project's core technology incorporates expertise in Mobile Application Development, ensuring an up-to-date and user-friendly solution for any individual from layman to a business professional seeking efficient travel experience.

Table of Contents:

Contents

Acknowledgement **0**

Abstract **2**

Comparison of Several Transport Systems: **5**

Chapter# 1: **7**

1.Introduction: **7**

1.1. Problem Statement: 7

1.2. Motivation: 7

1.3. Problem Definition: 8

1.4. Objective: 8

1.5. Scope: 8

1.6. System Definition: 9

1.7. Functionalities of app: 11

1.8. Constraints: 11

Chapter# 2 ..... **12**

2. Requirement and Analysis: **12**

2.1. Functional Requirements: 12

2.2. Non-Functional Requirements: 12

2.3. Use Case Diagram and Description: 13

2.3.1 Use Case Diagram: 13

2.3.2. Use Case Description: 16

Chapter# 3 35

3. System Design: 35

3.1. System Sequence Diagram: 36

3.2. Domain Model: 43

3.3. Sequence Diagrams: 44

3.4. Class Diagram: 51

Chapter# 4 53

4. Implementation: 53

4.1. Design Language (Main App): 53

4.2. Data Storage: 54

4.3. Web Based Admin Panel: 55

4.3.1 Design Language (Web): 56

4.3.2. Node JS: 57

Chapter# 5 58

5. Testing: 58

5.1. Testing Technique:.....58

5.2. Test Cases: 59

Chapter# 6:.....72

6. Conclusion: 72

6.1. Future Improvements: 73

6.2. References: 73

Comparison of Several Transport Systems:

I looked up the working of several transport systems for research purposes. Here is the comparison:

Features	Singapore Transport	Pakistan Railways	Indriver Pakistan
Real Time Locations	• Real time location	• No real time location tracking	• Real time location tracking
Payment Methods	• Smart cards • Online app • Mobile wallets • EZ link cards	• App • Website • Ticket booths	• Cash. • No online payment
Transport	• MRT (Mass Rapid Transit)	• Trains. • Long distance travels	• Cars • Bikes • Vans

Options	• LRT (Light Rail Transit) • SBS • Taxis.		• Travelling within the city.
Fares	• Separate Fares for Adults and Kids. • Customized Fares Options	• Fixed Fare for Adults. • Fixed Fare for Kids(separate)	• Fare Decided by company. • Remains Fixed.
Information Availability	• Details information like peak hours, peak periods, first and last trains etc.	• Good amount of information but not as much detailed.	• Valuable information about transport available.
Transport Sharing	• Transport sharing advanced.	• Limited transport sharing	• Transport sharing a key perk.
Accessibility	• Easily accessible all over the city.	• Limited accessibility features	• Easier than Railway systems but not Singapore transport system.
Discounted Options	• Special details for VIPs, groups, events etc	• Limited discount features	• Rare Discounts • Usually on first ride.

Chapter# 1:

1.Introduction:

In the dynamic environment of technological development, especially in context of public transportation, this project aims to address challenges encountered by users through the creation of a mobile application. The system strives to furnish users with comprehensive route information based on their specified origin and destination. Prioritizing routes based on specific criteria, the system is designed to improve the overall accessibility and efficiency of public transportation for users. This surely ensure marvelous travel experience and is completely user-friendly.

This chapter explains Problem Statement, Motivation, Problem Definitions, Objectives, Scope, System Definition, Functionalities and Constraints.

1.1. Problem Statement:

Nowadays it is very difficult for people to utilize public transportation due to lack of detailed information about routes, schedules, timings etc. This is the reason people prefer private transportation over local to avoid travel difficulties. There should be a system to facilitate people to utilize public transportation and plan their journeys smartly and effectively. This will save a lot of money and will encourage more people to utilize public transportation.

1.2. Motivation:

Locals often face challenges during their travel within the city. Though there are a lot of public transportation mediums available but the transportation medium that an average person uses the most has a lot of challenges namely missing route information, unknown arrival times, lack of fare information etc. So, my idea is to develop a user-friendly mobile application for all from layman to business professionals that will allow them to utilize the local transportation in the most efficient of ways. This app will allow them to get all the information related to their route,

fare, stops and will help in time management. Thus, this system will improve the means of public transportation and minimize complications for individuals.

1.3. Problem Definition:

Users of public transportation face difficulties now-a-days due to the insufficient availability of detailed and easily accessible information. Current systems often lack the provision of comprehensive data regarding the fastest routes, minimum fares and best possible vehicles changes, selection of appropriate vehicles etc. This informational deficiency impedes users from efficiently planning their journeys, resulting in a poor public transportation experience. In response to these challenges, the project seeks to create a mobile application that not only furnishes intricate route details but also prioritizes and filters routes based on specific criteria. This initiative aims to significantly improve the overall user-friendliness and efficacy of public transportation.

1.4. Objective:

An online user-friendly Mobile Application assisting in travel is much better than waiting for local transport that are not comprehensively known. This project has the main objective of filling this gap. This app will provide accurate details about routes, and relevant criteria for enhanced accessibility. The user interface will provide optimal routes based on minimum fares, minimal vehicle changes shortest travel paths based on user provided origin and destination. This will ensure cross-platform compatibility, allowing users access to various mobile devices for increased reach and accessibility.

1.5. Scope:

The scope of the project is to facilitate users in planning their journeys smartly. This involves creation of a mobile application to streamline public transportation for users with the following:

Features:

The application will have following features:

- Register
- Login
- Route Planning
    - Minimum vehicle changes.
    - Minimum fares.
    - Walking distances.
- Problem Report
- Profile Management

Technologies:

- Android Studio
- IntelliJ Idea
- Visual Studio Code

Languages:

- React Native
- Node JS
- Mongo DB
- EJS

Stakeholders:

- Admin/App Developer.
- Any User from a layman to Business professional.

1.6. System Definition:

The system will include:

Main Mobile Application:

The user will be able to access this mobile application for smartly planning their journeys through public transportation. The mobile application will provide the user will an environment where he/she can easily access the latest information for moving from one place to another.

The user will be able to perform the following:

- Register
- Login
- Validate Origin and Destination
- Plan routes and gain results based on following filters:
    - Minimum fares.
    - Minimum Vehicle Changes.
    - Walking distances.
- Submit Reports in case of errors for improvements.
- Profile Management.

Storage:

The database used for storing crucial information will be mongo DB. Mongo DB provides a flexible model with a lot of features for effective storage of data.

Following information will be stored in DB:

- Users' registration and login information.
- User's credentials.
- User's reports.
- Admin's login information.
- Admin side routes data.
- Admin side stops data.
- Admin side Paths data.
- Admin side fares data.
- Admin side route status.

1.7. Functionalities of app:

The app has following functionalities:

Register & Login:

- Users will be able to register.
- Once the registration is done, users can login and access the app.

Routes:

- Users will plan route based on origin and destination provided and optimal routes will be displayed.
- Routes will be ranked based on minimum vehicle change, minimum fares, shortest paths all including walking distances.

Feedback And Report:

- Users will be able to give feedback/report about the route information for further improvements.

1.8. Constraints:

Following are the constraints:

- Internet Connection should be available to the user.
- Users must be registered to login and ultimately use the app.
- The app shouldn't only be available on Android.
- User data must be protected with enhanced security.
- Route information should be accurate and precise.
- Layman should be able to access the app easily.
- Application should respond to requests quickly.

Chapter# 2

2. Requirement and Analysis:

This is a very crucial process as the complete project relies on good requirement analysis. Good requirement analysis is very important as it develops solutions of the problem and shapes the next phases of development.

2.1. Functional Requirements:

Functional Requirements are simply the functionalities the system provides that must be implemented so that the user can accomplish the desired tasks.

Following the functional requirements:

- Users will be able to register and then login into the application.
- Users will provide his/her origin (can be current location) and destination.
- Users will then be able to plan routes which will be filtered based on walking distances minimum fares, minimum vehicle changes etc.
- Users will be shown route which will be optimal for him.
- Users will be able to manage their profiles for managing their information.
- Users will be able to give feedback/report about routes for further improvements.

2.2. Non-Functional Requirements:

These depict the overall features, characteristics, and qualities that the app possess. Here are the non-functional requirements:

- **Security:**

  The application should provide complete security to protect user profiles, especially their data and location.

- **Usability:**

The application should be user-friendly and any layman from the street to any businessman should be able to use perfectly without any troubles.

- **Compatibility:**

  The application should be easily accessible on the latest android and iOS operating systems and should be fully responsive.

- **Performance:**

  The application should be fast and should respond to user requests within shortest time without any failures.

- **Reliability:**

  The application should be reliable, and it should have backup in case of server failures for maximum data integrity.

- **Availability:**

  The application must be available 24 hours a day without any troubles.

- **Scalability:**

  The application must be able to support multiple users in a single time frame without crashes and server failures.

2.3. Use Case Diagram and Description:

Use Case Diagram and Description are the essential parts of any project. They give an abstract representation of different functionalities and how the user interacts with them. Basically, it describes what the system is doing and how the actors will interact with the system. The diagram itself represents the functionality and scope of the system.

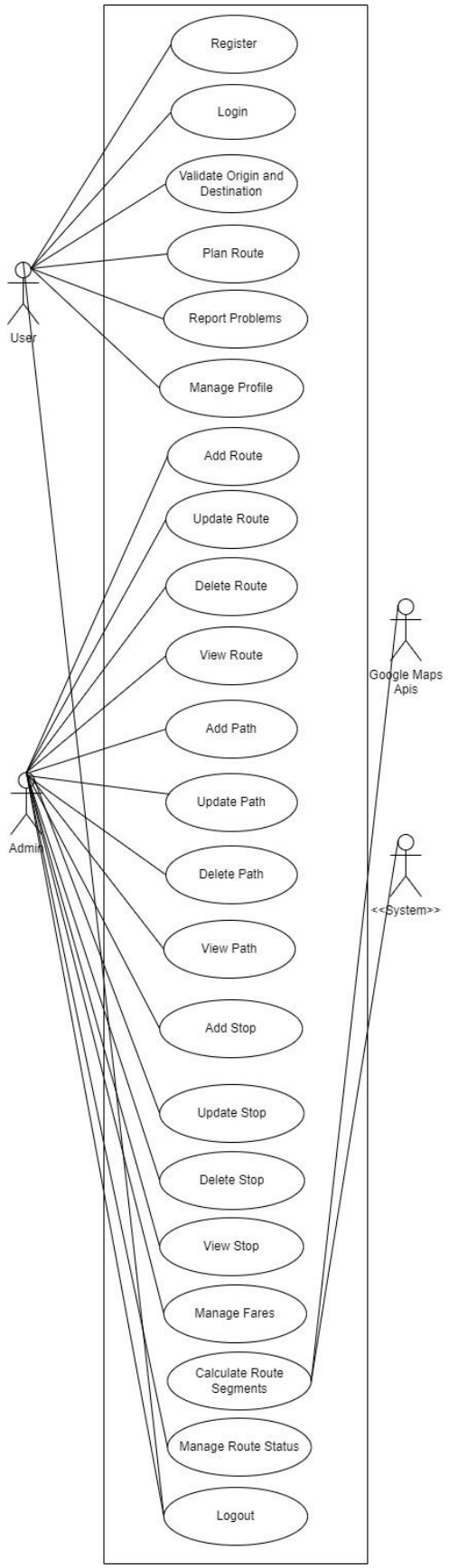
2.3.1 Use Case Diagram:

A use case diagram is a pictorial representation of how the user interacts with the system. It basically shows what the system does without the information of how it does it. Use case diagram describes the major functions along with the scope of the system. To summarize, it's a visual display of interaction of actors and the system.

The actors used in this diagram are:

- User
- Admin
- System
- Google Maps Apis

Following is the use case diagram for the intended application:


Register
Login
Validate Origin and Destination
Plan Route
Report Problems
Manage Profile
User
Add Route
Update Route
Delete Route
View Route
Add Path
Update Path
Delete Path
View Path
Add Stop
Update Stop
Delete Stop
View Stop
Manage Fares
Calculate Route Segments
Manage Route Status
Logout
Admin
Google Maps Apis
<<System>>

2.3.2. Use Case Description:

The use case description shows the details of each use case in term of pre-post conditions, actors, success scenarios etc.

Use Case ID	UC-1
Use Case Name	Register
Primary Actor	User
Secondary Actor	
Pre-Conditions	User has Internet Connection.
Post-Conditions	User successfully registers and is moved to login page.
Main Success Scenarios	1. User clicks the Register Button. 2. User Provides the registration details (Username, password). 3. User Information gets stored by the System. 4. The system takes the user to login screen after getting all details.

Alternate Scenarios	One of the following alternate scenarios can exist: 1. A meaningful error message is displayed if the user provides invalid or wrong information. 2. An account already exists with the same credentials.

Use Case ID	UC-2
Use Case Name	Login
Primary Actor	Admin, User
Secondary Actor	
Pre-Conditions	User is registered. User has internet connection.
Post-Conditions	User successfully logins and is directed to the home screen.
Main Success Scenarios	1. User clicks the login button. 2. User Provides the login details (Username, password.) and press login.

	3. System verifies the credentials and gives access. 4. User is directed to home screen.
Alternate Scenarios	One of the following alternate scenarios can exist 1. A meaningful error message is displayed if the user provides invalid or wrong information. 2. User resets the password by forget password option.

Use Case ID	UC-3
Use Case Name	Validate origin and destination.
Primary Actor	User
Secondary Actor	
Pre-Conditions	User is logged in. User has internet connection.
Post-Conditions	Valid origin and destination are entered.
Main Success Scenarios	1. User navigates to Search Screen. 2. User enters origin by following ways: 3. User turns on GPS. 3.1. User selects current location.

	4. User selects location from Search List. 5. User selects destination from Search list. 6. System validates the location. 7. System Displays markers on the origin and destination selected.
Alternate Scenarios	One of the following alternate scenarios can exist 1. A meaningful error message is displayed user enters invalid location (can be some other country or maybe some village where the service doesn't operate etc.) 2. System can't validate the location from google maps due to connectivity issues.

Use Case ID	UC-4
Use Case Name	Plan route
Primary Actor	User
Secondary Actor	
Pre-Conditions	User has provided valid origin and destination.

Post-Conditions	User views the planned route on google maps with transport details, fares etc.
Main Success Scenarios	1. User clicks the plan route button. 2. System displays the optimal route on the display. 2.1. Routes include walking distances (if required) 2.2. Mode changes (if required) 2.3. Transport details. 2.4. Overall possible fares etc.
Alternate Scenarios	A meaningful error message is displayed as system can't provide route information due to connectivity issues.

Use Case ID	UC-5
Use Case Name	Add Route
Primary Actor	Admin
Secondary Actor	
Pre-Conditions	User has logged in.
Post-Conditions	User has successfully added a new route to the database.

Main Success Scenarios	1. User goes to add route option. 2. User adds a new route (Name, Status, Path). 3. A new route is added by the user. 4. The database is updated.
Alternate Scenarios	A meaningful message is displayed if user couldn't connect to database to add the route.

Use Case ID	UC-6
Use Case Name	Update Route
Primary Actor	Admin
Secondary Actor	
Pre-Conditions	User has logged in.
Post-Conditions	User has successfully updated the route in the database.
Main Success Scenarios	1. User finds the route to update.

	2. User clicks on update route option. 3. User updates the route with new information (Name, Path, Status). 4. The selected route is updated. 5. The database is updated.
Alternate Scenarios	A meaningful message is displayed if the user couldn't connect to database to update the route.

Use Case ID	UC-7
Use Case Name	Delete Route
Primary Actor	Admin
Secondary Actor	
Pre-Conditions	User has logged in.
Post-Conditions	User has successfully deleted the route (ID)
Main Success Scenarios	1. User finds the route to delete. 2. User goes to delete route option. 3. User deletes the route. 4. All information linked to this route gets deleted. 5. The database is updated.

Alternate Scenarios	A meaningful message is displayed if the user can't connect to database to delete the route.

Use Case ID	UC-8
Use Case Name	View Route
Primary Actor	Admin
Secondary Actor	
Pre-Conditions	User has logged in.
Post-Conditions	User has successfully searched a route(ID)
Main Success Scenarios	1. User finds the route to view. 2. User clicks on view route option. 3. The information related to that route is displayed to the user.
Alternate Scenarios	A meaningful message is displayed if the user can't connect to database to the route.

Use Case ID	UC-9
Use Case Name	Add Stop
Primary Actor	Admin
Secondary Actor	
Pre-Conditions	User has logged in.
Post-Conditions	User has successfully added stops information to the database.
Main Success Scenarios	1. User goes to add stop option. 2. User adds stop information (name, stop's lat & long). 3. The database is updated.
Alternate Scenarios	A meaningful message is displayed if user couldn't add stop info to database due to connectivity issue.

Use Case ID	UC-10

Use Case Name	Update Stop
Primary Actor	Admin
Secondary Actor	
Pre-Conditions	User has logged in.
Post-Conditions	User has successfully updated stops information from the database.
Main Success Scenarios	1. User finds the stop he wants to update. 2. User goes to update stop option. 3. User updates stop information (name, stop's lat & long). 4. The stop information is updated. 5. The database is updated.
Alternate Scenarios	A meaningful message is displayed if the user can't update stop info to database due to connectivity issue.

Use Case ID	UC-11
Use Case Name	Delete Stop
Primary Actor	Admin

Secondary Actor	
Pre-Conditions	User has logged in.
Post-Conditions	User has successfully deleted stops information from the database.
Main Success Scenarios	• User finds the stop he wants to delete. • User deletes the stop (ID) information. • All information related to that stop is removed. • The database is updated.
Alternate Scenarios	A meaningful message is displayed if the user can't delete stop info from database due to connectivity issue.

Use Case ID	UC-12
Use Case Name	View Stop
Primary Actor	Admin
Secondary Actor	
Pre-Conditions	User has logged in.

Post-Conditions	User has successfully viewed some stop's information.
Main Success Scenarios	• User goes to view stop (ID) option. • The stops information is displayed to the user.
Alternate Scenarios	A meaningful message is displayed if user can't connect to database to view stop information.

Use Case ID	UC-13
Use Case Name	Add Path
Primary Actor	Admin
Secondary Actor	
Pre-Conditions	User has logged in.
Post-Conditions	User has successfully added path information.
Main Success Scenarios	• User goes to add path option. • The user adds the path information i-e Path name, Stops, Stop number etc. • The information is saved to the database.

Alternate Scenarios	A meaningful message is displayed if user can't connect to database add path information.

Use Case ID	UC-14
Use Case Name	Update Path
Primary Actor	Admin
Secondary Actor	
Pre-Conditions	User has logged in.
Post-Conditions	User has successfully updated path information to the database.
Main Success Scenarios	1. User finds the path he wants to update. 2. User goes to update path option. 3. User updates path information (name, stops, stops number). 4. The path information is updated to the database
Alternate Scenarios	A meaningful message is displayed if the user can't update path info to database due to connectivity issue.

Use Case ID	UC-15
Use Case Name	Delete Path
Primary Actor	Admin
Secondary Actor	
Pre-Conditions	User has logged in.
Post-Conditions	User has successfully deleted path information from the database.
Main Success Scenarios	1. User finds the path he wants to delete. 2. User goes to delete path option. 3. User deletes path information (ID). 4. The path information is deleted from the database.
Alternate Scenarios	A meaningful message is displayed if the user can't delete path info from database due to connectivity issue.

Use Case ID	UC-16
Use Case Name	View Path

Primary Actor	Admin
Secondary Actor	
Pre-Conditions	User has logged in.
Post-Conditions	User has successfully viewed path information from the database.
Main Success Scenarios	1. User finds the path he wants to view. 2. User goes to view path option. 3. User views path information (ID). 4. The path information is displayed on the screen from the database.
Alternate Scenarios	A meaningful message is displayed if the user can't view path information from database due to connectivity issue.

Use Case ID	UC-17
Use Case Name	Manage Fares
Primary Actor	Admin
Secondary Actor	

Pre-Conditions	User is logged in.
Post-Conditions	User has successfully managed fares in the database.
Main Success Scenarios	1. User finds the route to route to manage its fare. 2. User manages the fare. 3. The changes are saved to the database.
Alternate Scenarios	A meaningful message is displayed if user couldn't manage fares in the database due to connectivity issue.

Use Case ID	UC-18
Use Case Name	Calculate route segments
Primary Actor	System
Secondary Actor	Google Map Api
Pre-Conditions	A request for route planning is made.
Post-Conditions	Route is successfully divided into segments.
Main Success Scenarios	1. The system retrieves necessary information from the database.

	2. System uses google maps Api to calculate walking distance (if required) 3. The system uses google maps Api to calculate distance between stops etc. 4. System provides the segmented path for display to the user of the app.
Alternate Scenarios	A meaningful message is displayed if system couldn't provide proper dissection of path due to connectivity issue.

Use Case ID	UC-19
Use Case Name	Manage Route Status
Primary Actor	Admin
Secondary Actor	
Pre-Conditions	User is logged in.
Post-Conditions	Route status has been managed successfully in the database.
Main Success Scenarios	1. User goes to the route option. 2. User selects the route he wants to manage. 3. User specifies whether the route is active or temporarily non-active. 4. The changes are saved in the database.

Alternate Scenarios	A meaningful message is displayed if user couldn't set route status due to connectivity issue.

Use Case ID	UC-20
Use Case Name	Report Problems
Primary Actor	User
Secondary Actor	
Pre-Conditions	Problem exists with the routes.
Post-Conditions	1. Users successfully reported problems with the route. 2. Report is sent to admin for analysis.
Main Success Scenarios	1. User has no route generated for him. 2. User navigates to report section. 3. Users write the report/feedback. (remarks about the route). 4. Report is sent to admin for analysis.
Alternate Scenarios	No problems exist with the route.

Use Case ID	UC-21
Use Case Name	Manage Profile
Primary Actor	User
Secondary Actor	
Pre-Conditions	User is logged in.
Post-Conditions	User successfully manages his profile.
Main Success Scenarios	1. User changes his credentials (Username, password etc.) 2. User customizes the interface for themes (optional).
Alternate Scenarios	User has trouble customizing profile.

Use Case ID	UC-22
Use Case Name	Logout
Primary Actor	User, Admin

Secondary Actor	
Pre-Conditions	User is logged in.
Post-Conditions	User logs out of the system.
Main Success Scenarios	1. User presses the logout button. 2. The control is back to the login screen.
Alternate Scenarios	Internet connection was interrupted during the process and logout procedure couldn't complete.

Chapter# 3

3. System Design:

System design is the initial phase before implementing a system. It involves defining the architecture, modules, interfaces, and data flow to meet specific requirements. It visually depicts how different components/modules interact to provide desired functionality.

It has following parts:

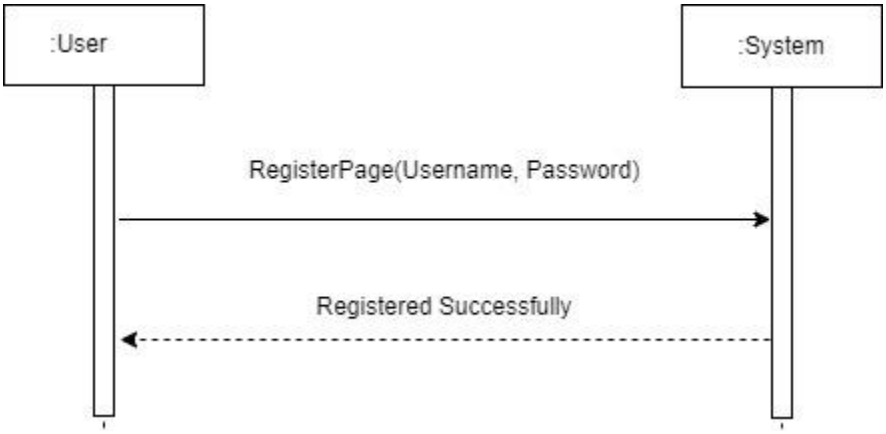
- System Sequence Diagram
- Domain Model
- Sequence Diagram
- Class diagram

3.1. System Sequence Diagram:

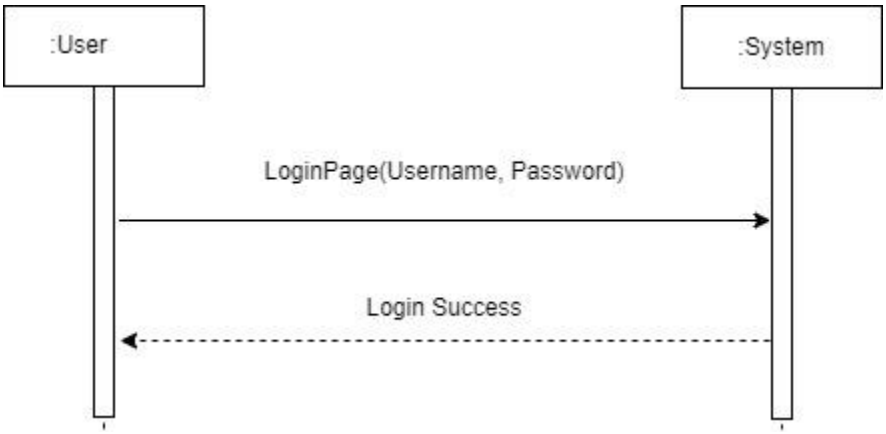
A system sequence diagram is a pictorial representation of interaction between user and the system. It illustrates the flow of messages or actions exchanged over time to achieve specific functionality within the system. SSDs can play a crucial role in design, analysis and communication of systems which allow stakeholders to interact with each other effectively.

Here are the SSDs:

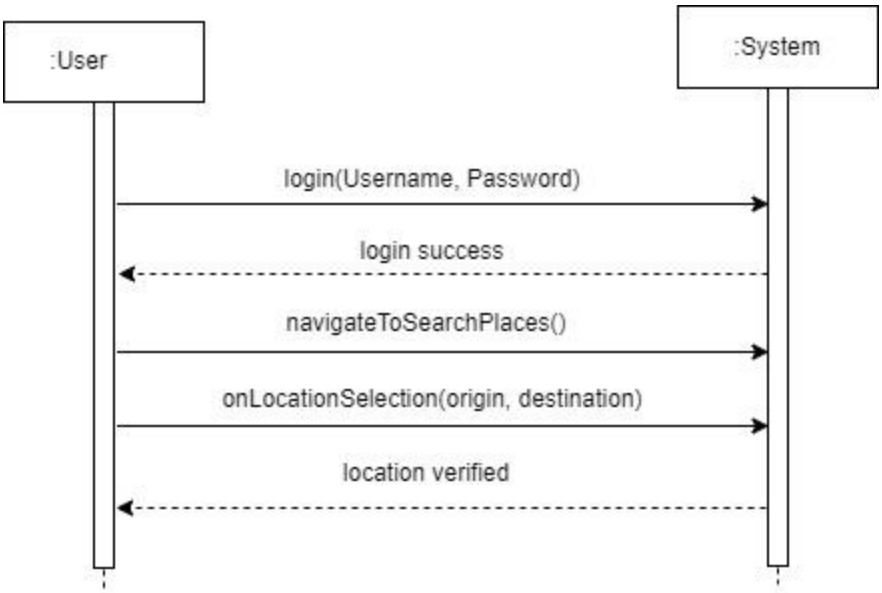
Registration:



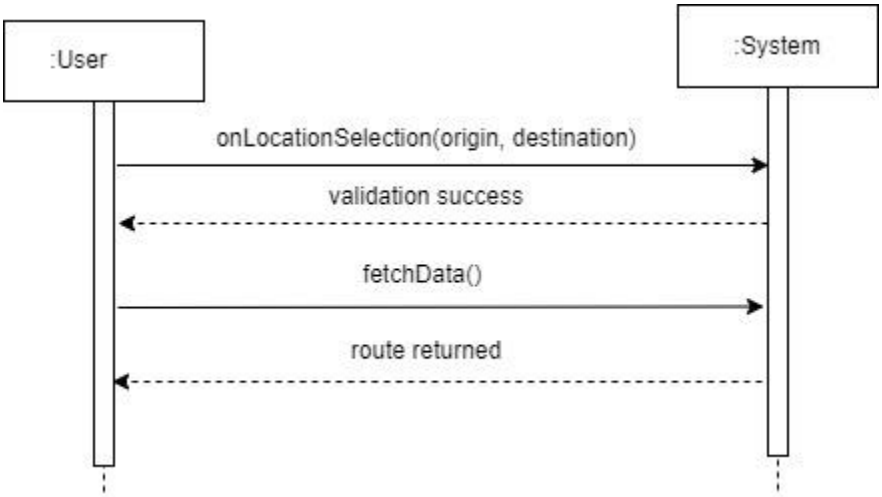
Login:



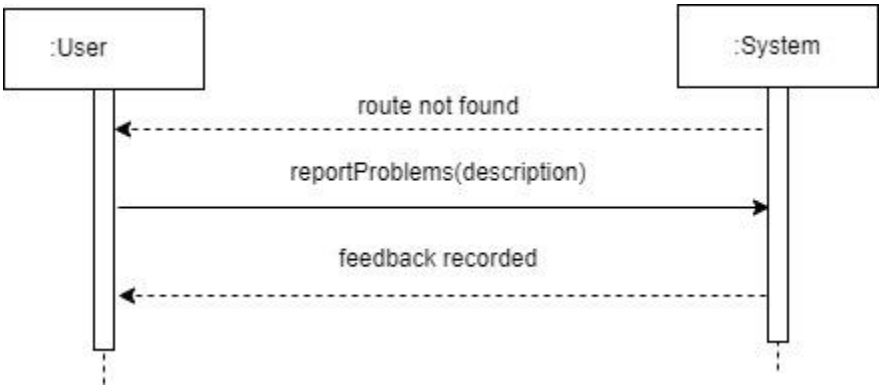
Validate Origin and Destination:


:User
:System
login(Username, Password)
login success
navigateToSearchPlaces()
onLocationSelection(origin, destination)
location verified

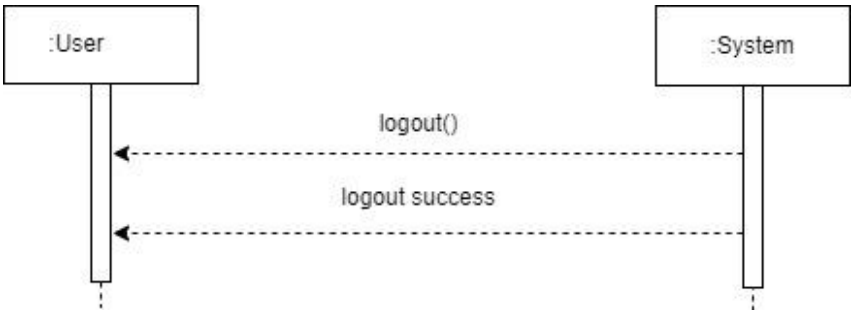
Plan Route:


:User
:System
onLocationSelection(origin, destination)
validation success
fetchData()
route returned

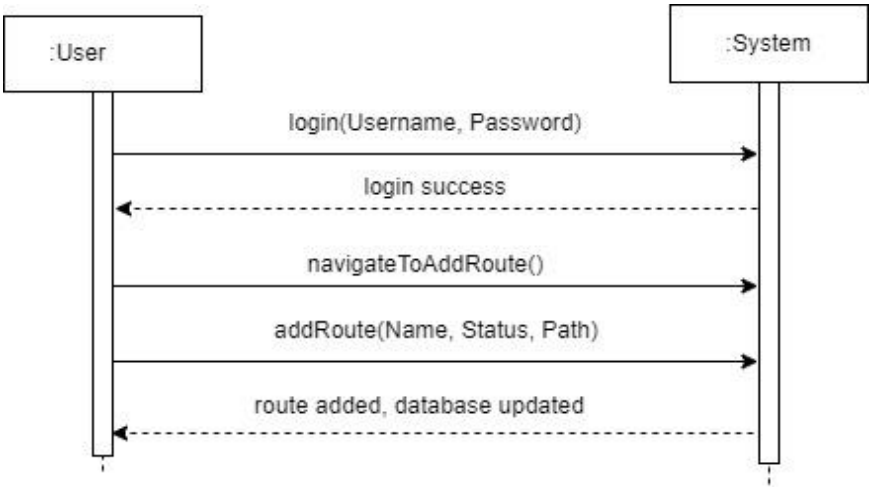
Report Problems:



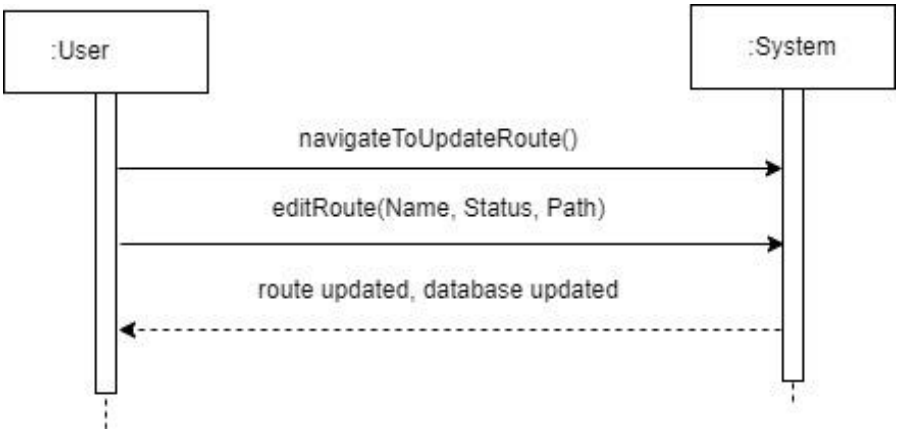
Logout:



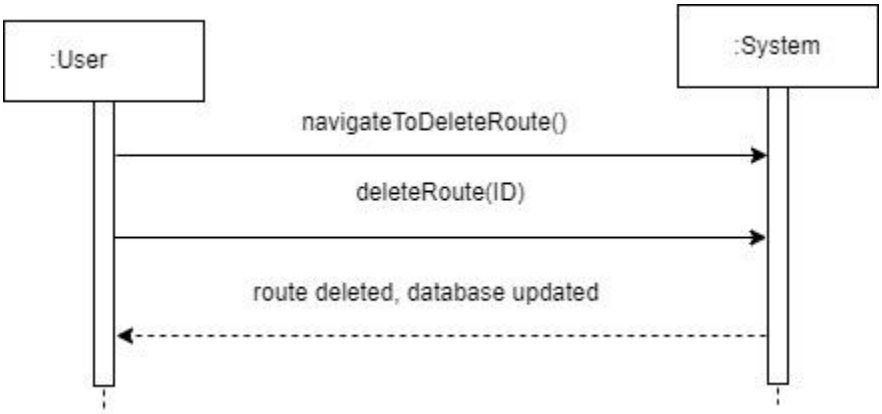
Add Route:



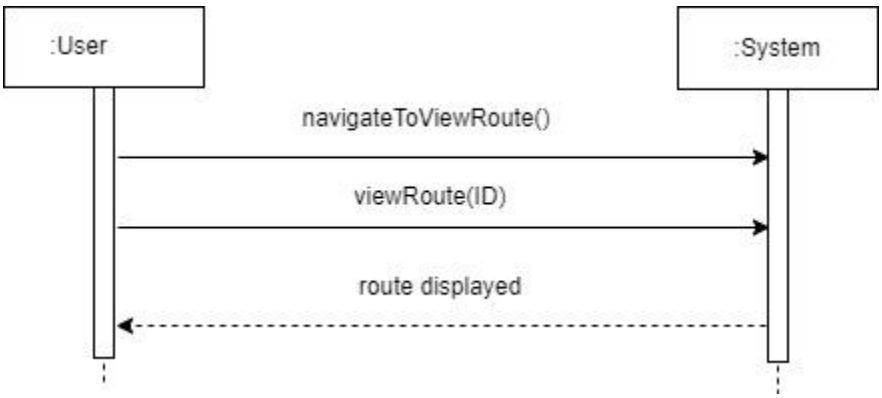
Update Route:



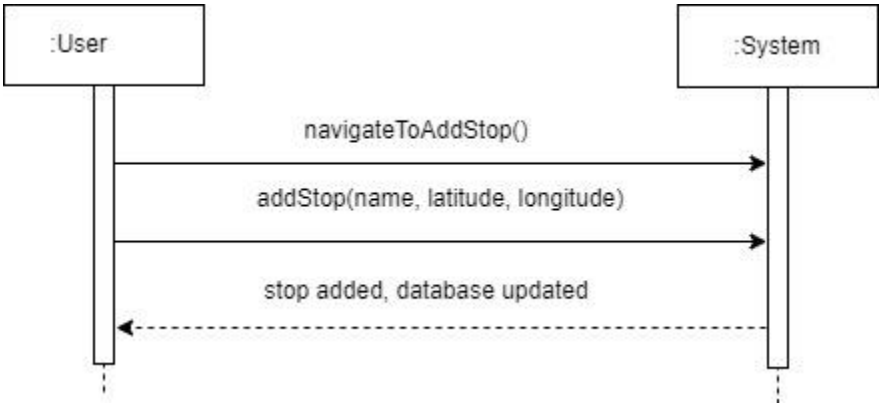
Delete Route:



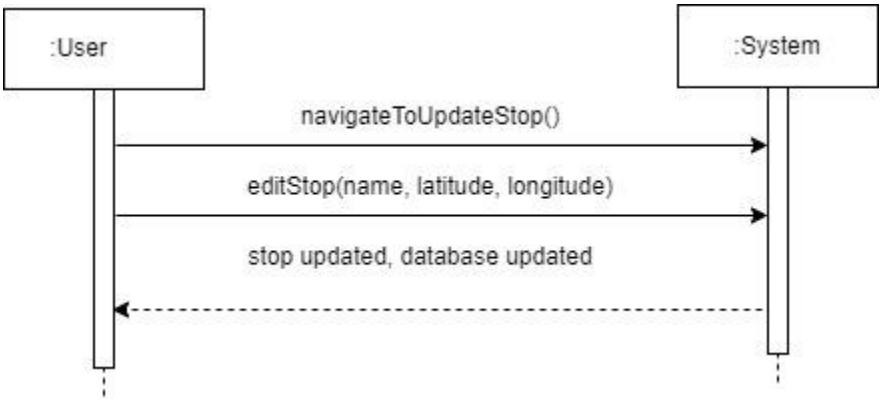
View Route:



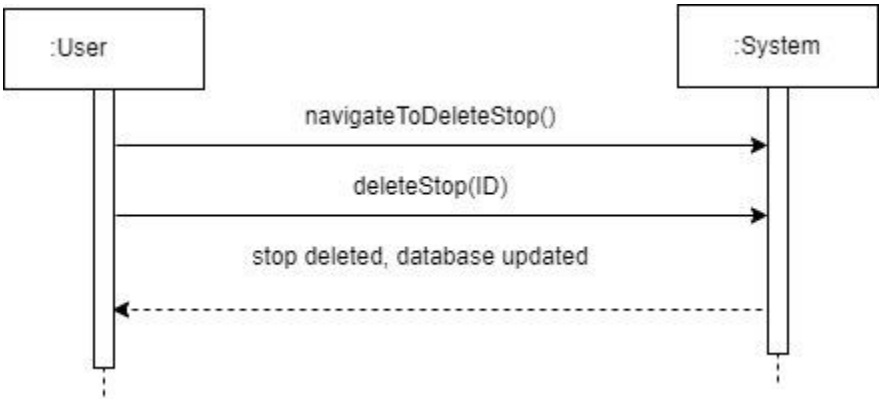
Add Stop:



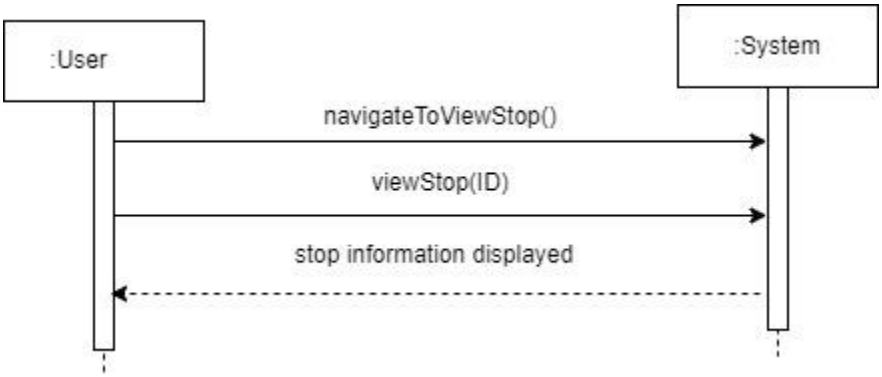
Update Stop:



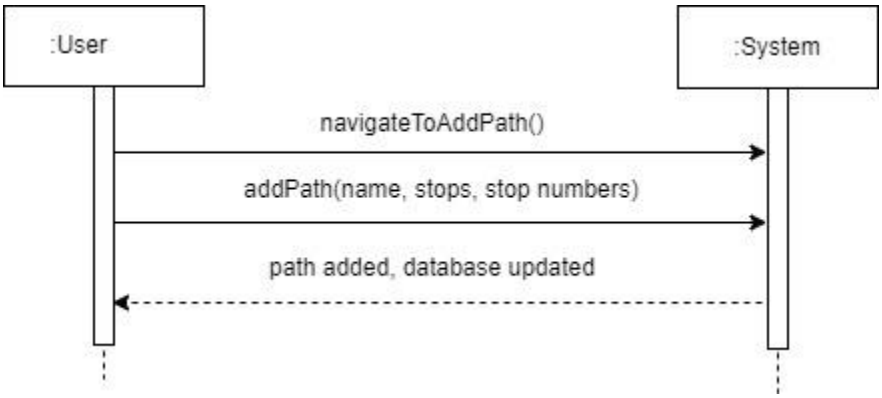
Delete Stop:



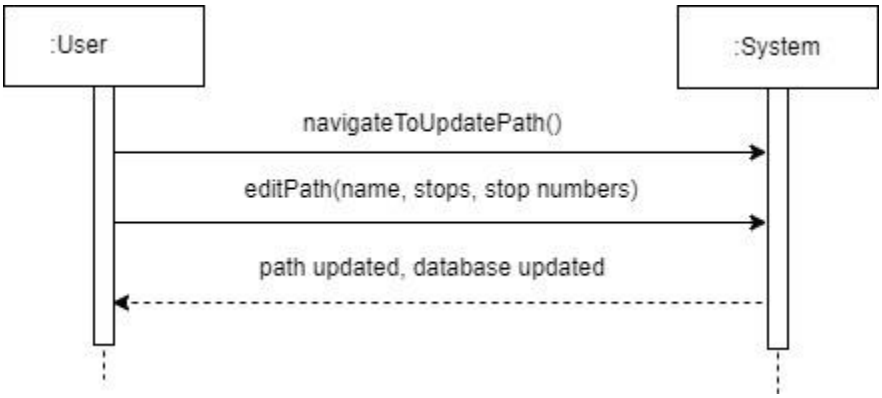
View Stop:



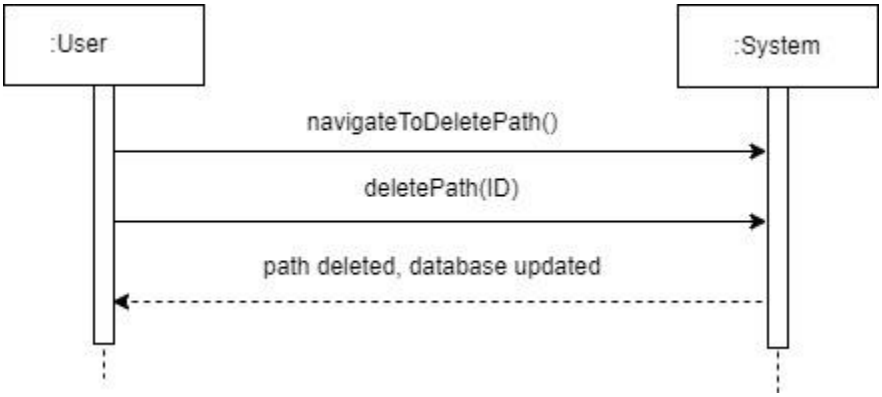
Add Path:



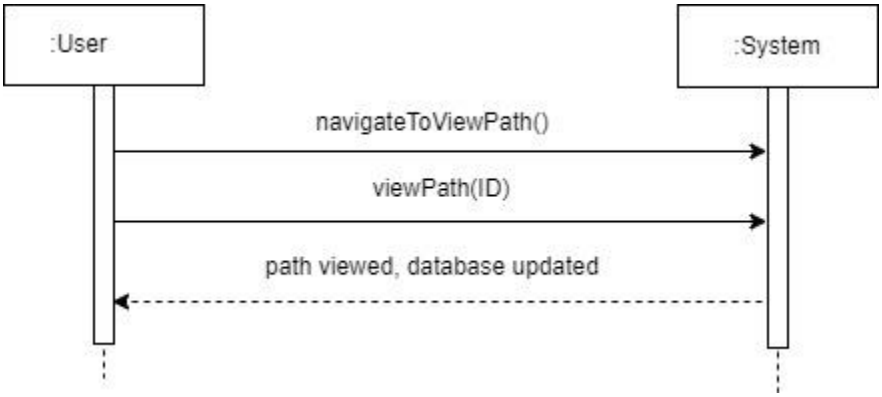
Update Path:



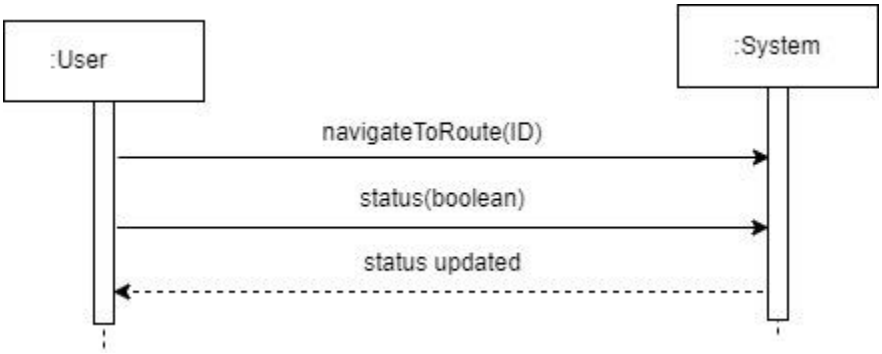
Delete Path:



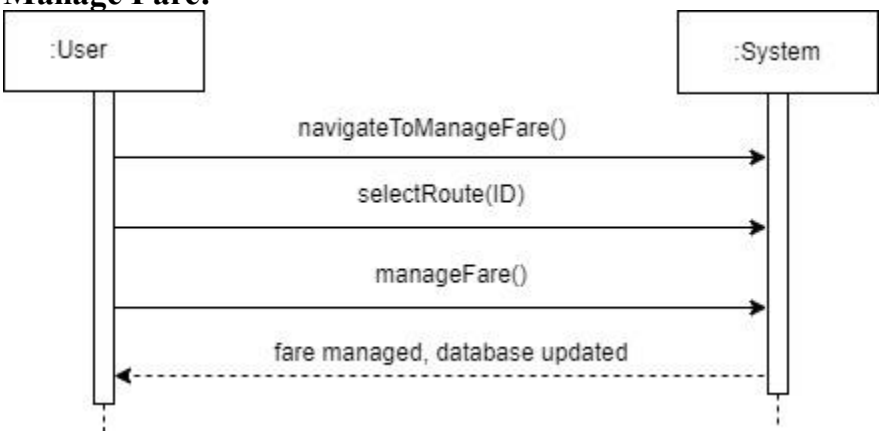
View Path:



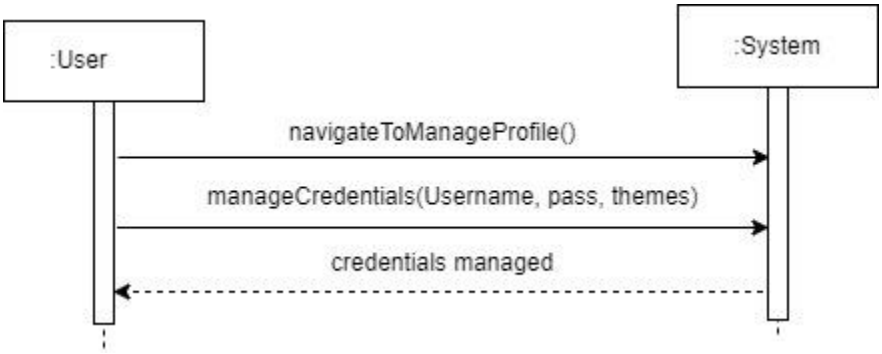
Manage route status:



Manage Fare:



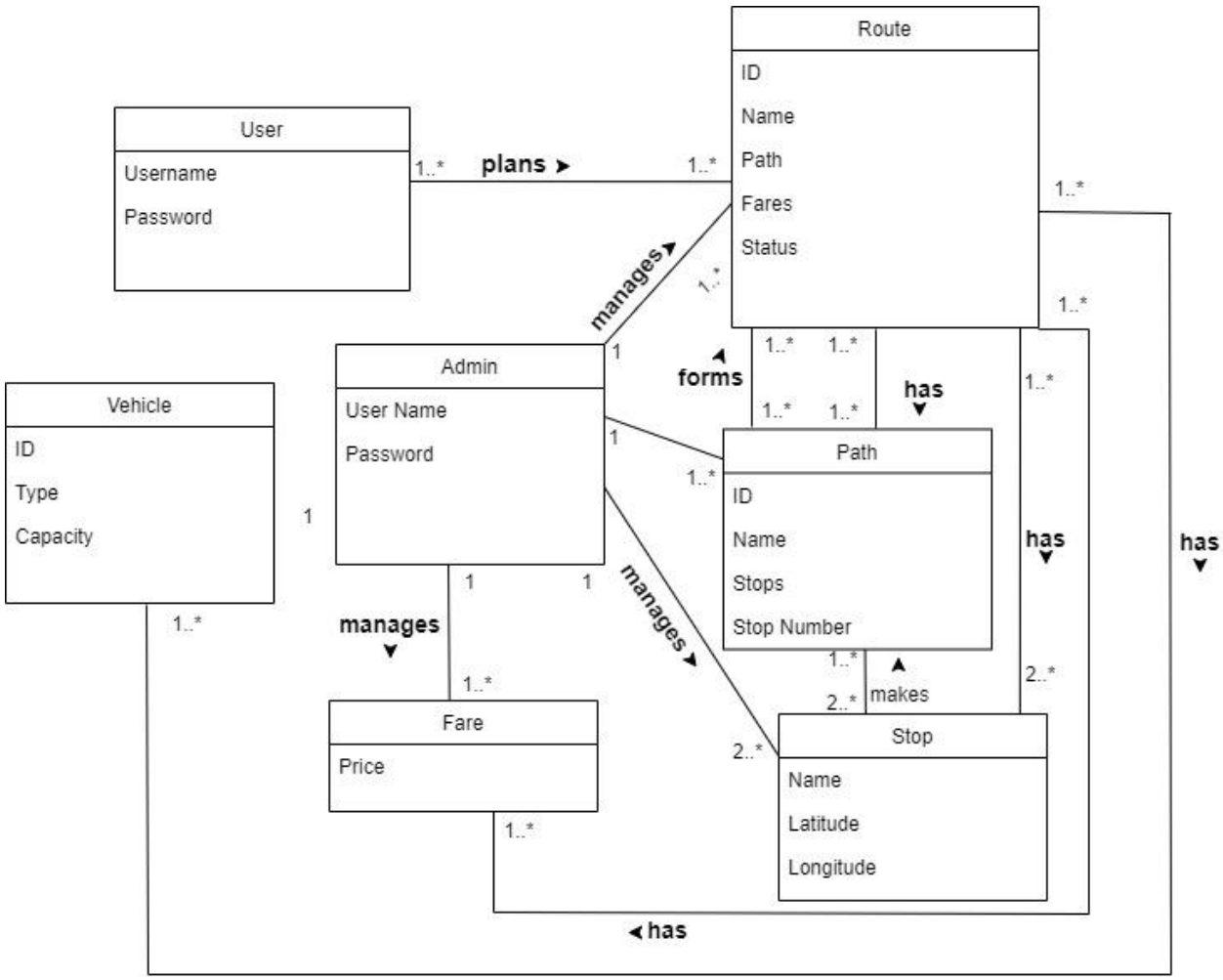
Manage Profile:



3.2. Domain Model:

A domain model is a problem domain object model. Domain model is a pictorial representation of real-world entities with their attributes along with the relationships between them. Domain model is an important process during the design phase that represents key step of the core problem's domain.

Following is the domain model representation:

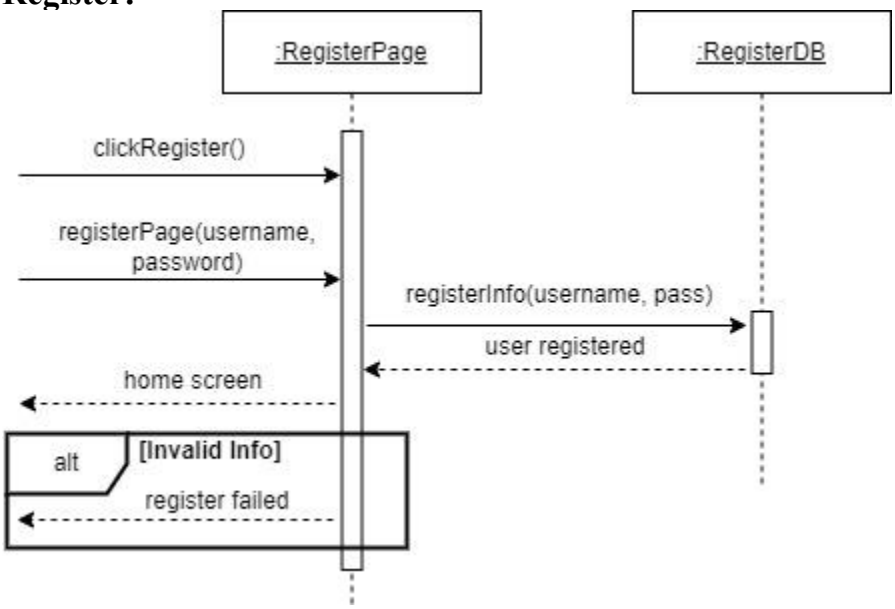


3.3. Sequence Diagrams:

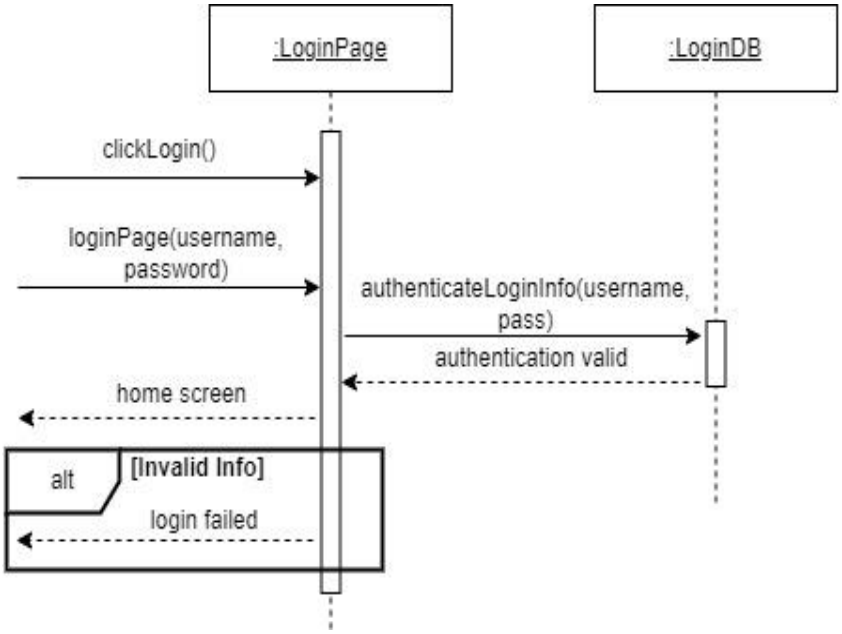
A sequence diagram is a pictorial representation of sequence of interactions between objects. It shows the proper sequence of exchange of messages over time between objects as required to carry out proper functionality.

Following are sequences diagrams showing the sequence of interactions of objects:

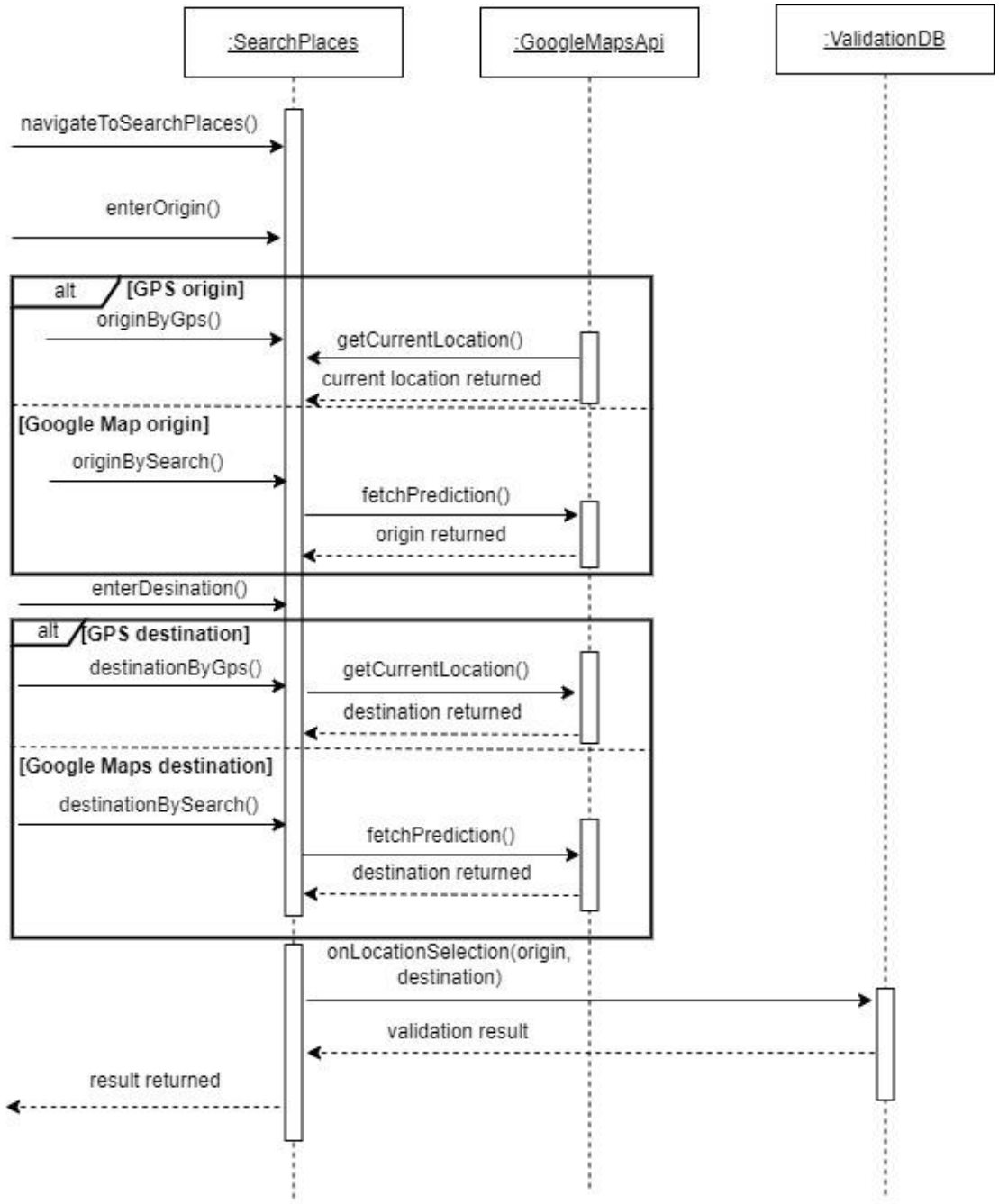
Register:



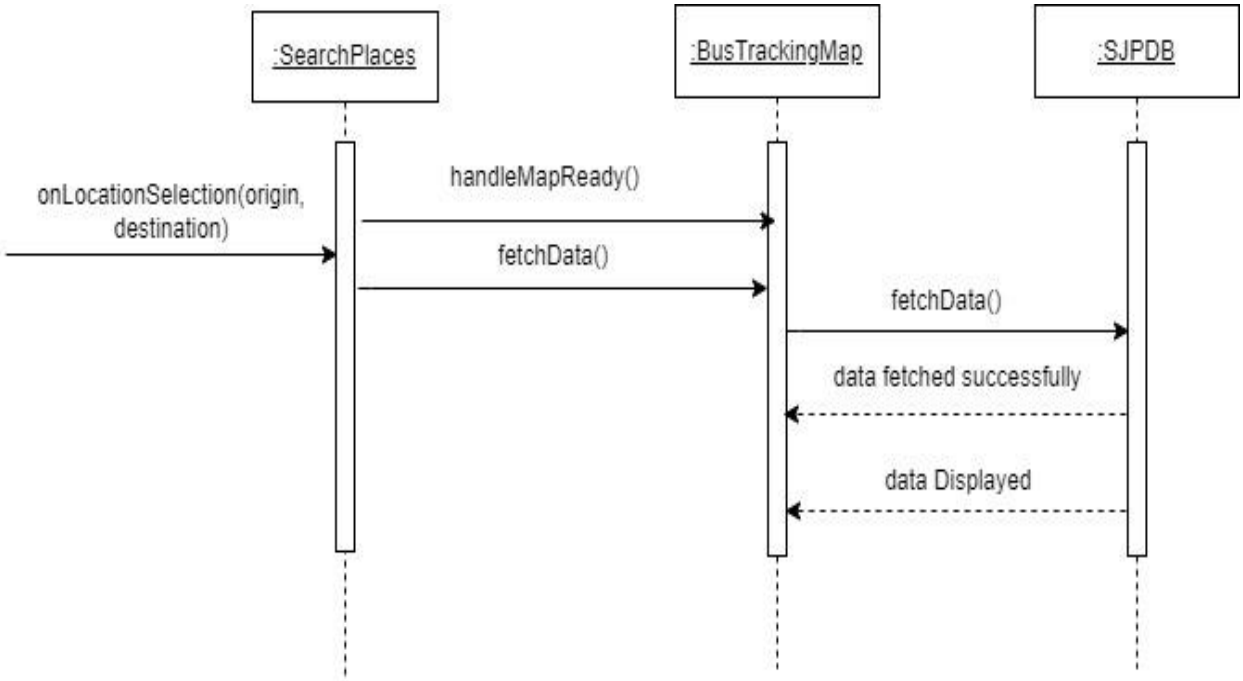
Login:



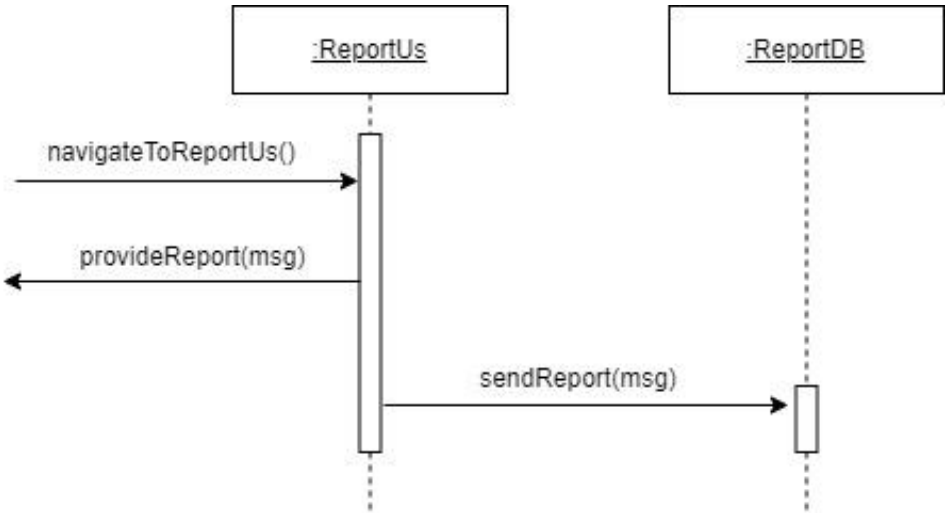
Location Validation:



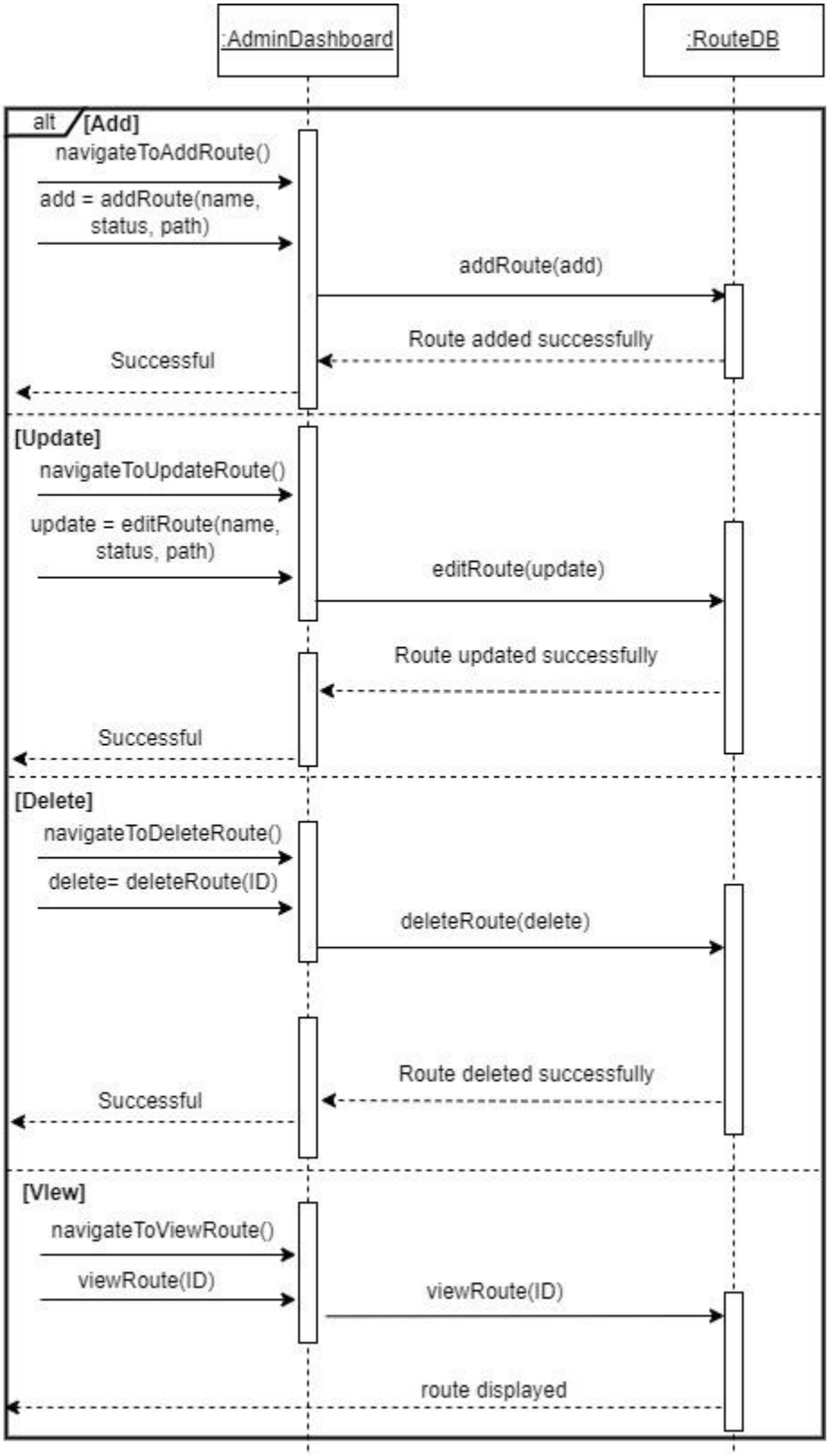
Plan route:



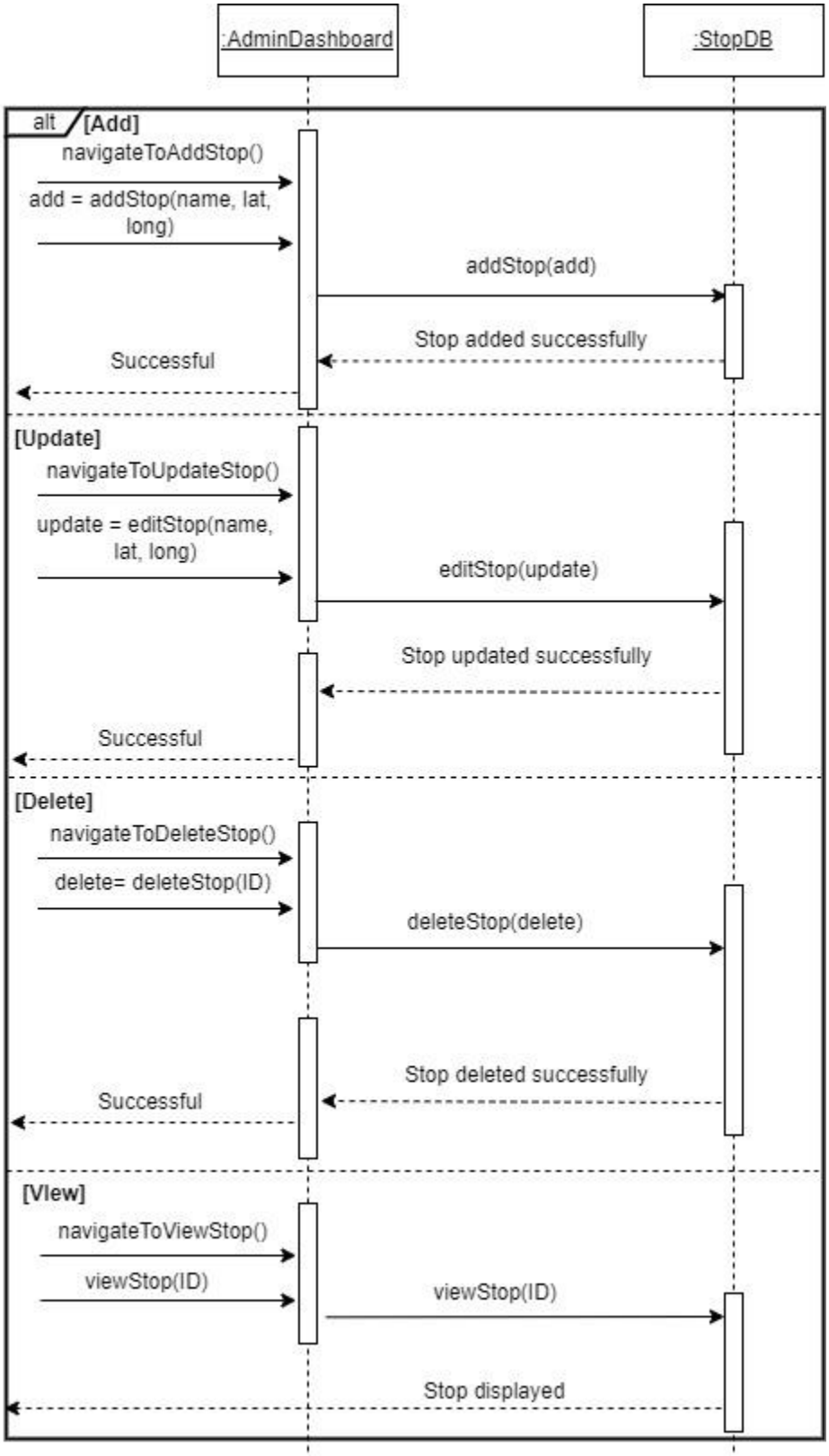
Report Us:



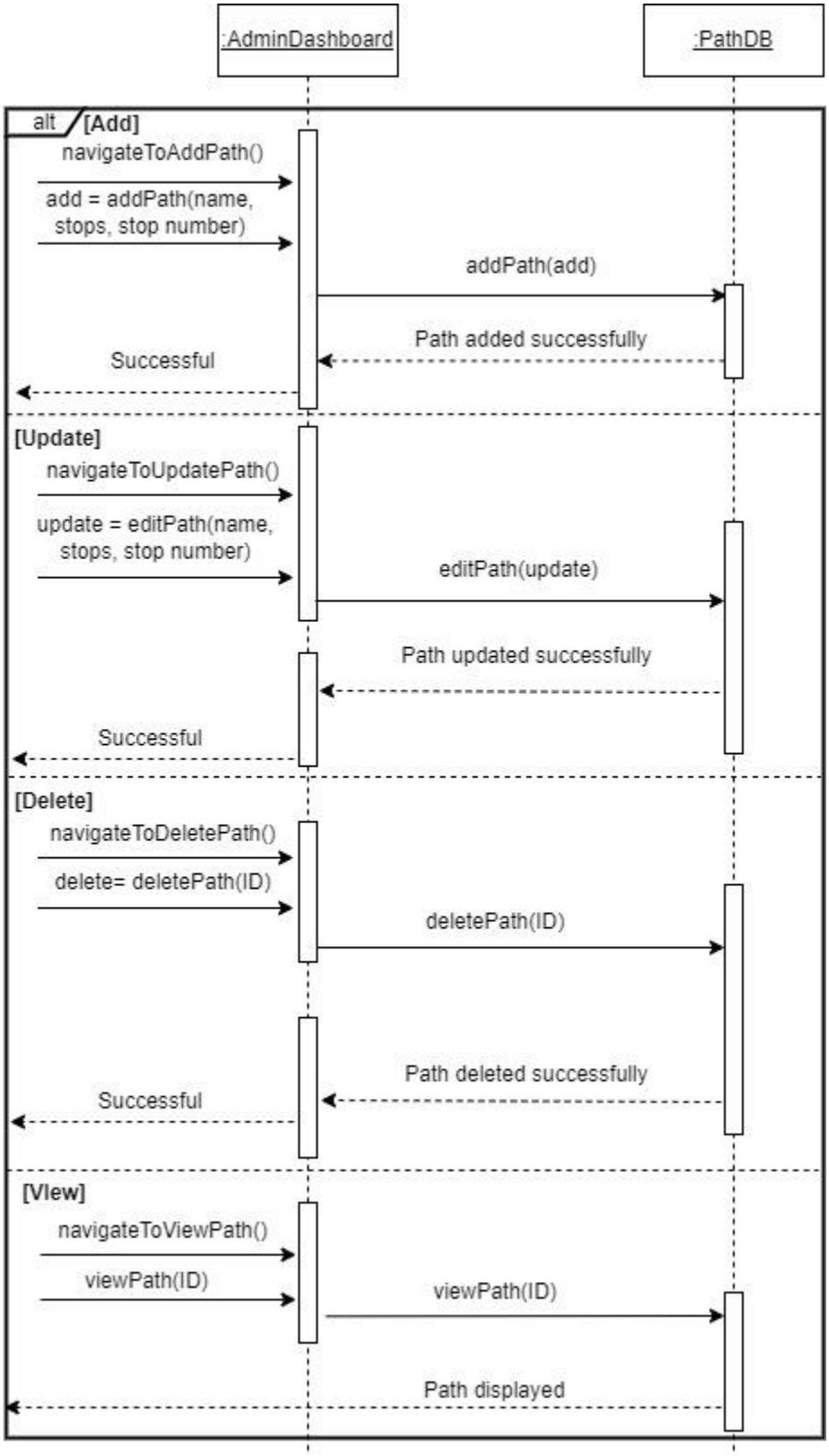
Manage Routes:



Manage Stop:


:AdminDashboard
:StopDB
alt
[Add]
navigateToAddStop()
add = addStop(name, lat, long)
addStop(add)
Stop added successfully
Successful
[Update]
navigateToUpdateStop()
update = editStop(name, lat, long)
editStop(update)
Stop updated successfully
Successful
[Delete]
navigateToDeleteStop()
delete= deleteStop(ID)
deleteStop(delete)
Stop deleted successfully
Successful
[View]
navigateToViewStop()
viewStop(ID)
viewStop(ID)
Stop displayed

Manage Path:


:AdminDashboard
:PathDB
alt
[Add]
navigateToAddPath()
add = addPath(name, stops, stop number)
addPath(add)
Path added successfully
Successful
[Update]
navigateToUpdatePath()
update = editPath(name, stops, stop number)
editPath(update)
Path updated successfully
Successful
[Delete]
navigateToDeletePath()
delete= deletePath(ID)
deletePath(ID)
Path deleted successfully
Successful
[View]
navigateToViewPath()
viewPath(ID)
viewPath(ID)
Path displayed

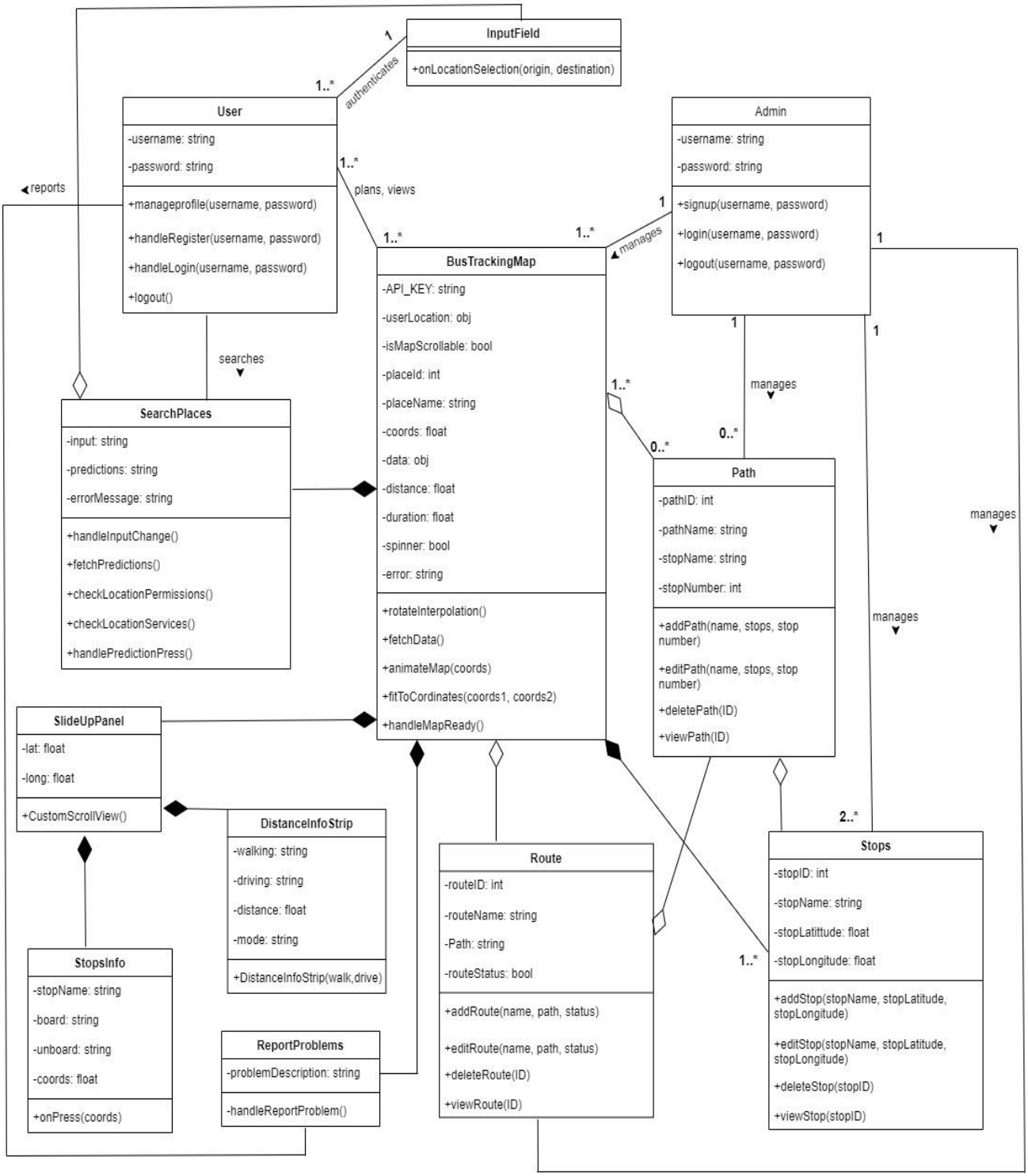
3.4. Class Diagram:

A class diagram is a UML diagram that pictorially represents structure and relationship between different software classes by defining attributes, operations, and relationships between them. Class diagram is a crucial part of design phase that basically represents the anatomy of the coding phase by defining major classes and functionalities.

Class diagram has the following:

- Classes
- Attributes
- Methods
- Relationships
- Visibility
- Interfaces etc

Following is the class diagram representing classes, attributes, and relationship between them:


InputField
+onLocationSelection(origin, destination)
authenticates
1..*
User
-username: string
-password: string
+manageprofile(username, password)
+handleRegister(username, password)
+handleLogin(username, password)
+logout()
reports
1..*
plans, views
1..*
1..*
BusTrackingMap
-API_KEY: string
-userLocation: obj
-isMapScrollable: bool
-placeId: int
-placeName: string
-coords: float
-data: obj
-distance: float
-duration: float
-spinner: bool
-error: string
+rotateInterpolation()
+fetchData()
+animateMap(coords)
+fitToCordinates(coords1, coords2)
+handleMapReady()
manages
1
Admin
-username: string
-password: string
+signup(username, password)
+login(username, password)
+logout(username, password)
1
searches
SearchPlaces
-input: string
-predictions: string
-errorMessage: string
+handleInputChange()
+fetchPredictions()
+checkLocationPermissions()
+checkLocationServices()
+handlePredictionPress()
1..*
0..*
0..*
manages
1
1
Path
-pathID: int
-pathName: string
-stopName: string
-stopNumber: int
+addPath(name, stops, stop number)
+editPath(name, stops, stop number)
+deletePath(ID)
+viewPath(ID)
manages
manages
SlideUpPanel
-lat: float
-long: float
+CustomScrollView()
DistanceInfoStrip
-walking: string
-driving: string
-distance: float
-mode: string
+DistanceInfoStrip(walk,drive)
Route
-routeID: int
-routeName: string
-Path: string
-routeStatus: bool
+addRoute(name, path, status)
+editRoute(name, path, status)
+deleteRoute(ID)
+viewRoute(ID)
2..*
1..*
Stops
-stopID: int
-stopName: string
-stopLatittude: float
-stopLongitude: float
+addStop(stopName, stopLatitude, stopLongitude)
+editStop(stopName, stopLatitude, stopLongitude)
+deleteStop(stopID)
+viewStop(stopID)
StopsInfo
-stopName: string
-board: string
-unboard: string
-coords: float
+onPress(coords)
ReportProblems
-problemDescription: string
-handleReportProblem()

Chapter# 4

4. Implementation:

Journey planner for public transportation is a mobile based application that facilitates the users ranging from layman to business professionals to plan their journeys effectively and ensure seamless travel experience. This application will provide the user with functionalities such as:

- The user can register and login in the application.
- The user can validate their origin and destination to check whether the services operate within the origin and destination provided.
- The user can plan route and will have optimal route available to him.
- Users can report problems if there are any with the routes.
- Users can manage their profiles for themes, usernames, passwords etc.

4.1. Design Language (Main App):

The mobile application is designed by React Native which is a hybrid open-source software framework designed by Meta platforms. It can effectively build rich mobile applications using JavaScript and React. It is widely used to develop mobile applications and is preferred due to the following reasons:

Hybrid nature:

React native is a hybrid framework. This allows a single generation of code which can be deployed for both android and iOS-based applications. This practice saves a lot of time and effort and allows the developer to make effective code without the need to generate it for different platforms.

Budget Effectiveness:

React native applications require a single generation of code hence saving a lot of cost and effort. The resources required for the development, deployment and maintenance are much less cost effective than using other development means.

Code Reusability:

React native enables users to create reusable components which can be shared/exchanged between different platforms. This sharing of reusable components can help attain speed and code reusability to a high level. Thus, the development, deployment and maintenance process become highly effective.

Hot Reloading:

React native allows a feature of hot reloading meaning that the changes made in the code can be reflected in real time without the need to refresh the app. This maintains the state of the app and helps a lot in debugging processes and accelerates development and maintenance.

Learning Effectiveness:

React native is easier to learn and implement. A user with basic react knowledge can also transition to react native development without putting much effort. Hence react native provides smooth development experience within less effort.

Third Party Plugin:

React native promotes the use of third-party plugins and libraries which developers can use to incorporate different functionalities i-e map locations, real time notifications etc. Hence the capabilities of these plugins are improved.

User experience:

React native allows the user to have smooth user experience by using different features. Certain features enable the user to improve his overall performance with enhanced satisfaction.

4.2. Data Storage:

The data is stored in mongo DB which provides a lot of ease in terms of the following features:

Flexible Model:

Mongo DB provides a flexible model for storing data in the database providing the users with the special privilege of storing data in BJSON (Binary JavaScript Object Notation) formats. The advantage of such formats is to represent complex structures with ease.

Performance:

Mongo DB has features like query optimization, indexing etc that enables high speed and efficient data addition, retrieval, searching and manipulation. Thus, the overall performance is largely improved.

User Experience:

Mongo DB provides smooth user experience. The interface provided by mongo DB is user-friendly. APIs can easily be used with mongo DB for easier access to different real time data. Moreover, the integration into existing mobile applications is effective.

Data Scalability:

Ample amount of data can easily be handled using mongo DB. It helps control humps of loaded data which can also be distributed across multiple devices. Redundant data, fault management etc are some features that come with scalability.

Query Enrichment:

Mongo DB provides enriched query features such as managing, handling, filtering, and sorting complex queries effectively. The data addition, retrieval and manipulation are improved with such features.

4.3. Web Based Admin Panel:

Journey planner mobile application requires retrieval of data from database specifically mongo DB. The admin on the other hand requires a web-based application/panel to store details to the DB for efficient retrieval in the mobile application. The web-based application/panel has the following functionalities:

- Admin will add, update, search, delete and view route information.
- Admin will add, update, search, delete and view stop information.
- Admin will add, update, search, delete and view path information.
- Admin will manage fares information.
- Admin will manage route status for active and inactive routes.

4.3.1 Design Language (Web):

The web application will be designed using EJS for front-end development and Node JS for backend development while the database remains the same (Mongo DB).

1. Easy to Use

EJS is straightforward and easy to learn, making it a great choice for developers who are already familiar with HTML and JavaScript. Its syntax is simple, and it integrates seamlessly with Node.js applications.

2. Dynamic HTML Rendering

EJS allows for dynamic HTML rendering. You can embed JavaScript code within your HTML to create dynamic web pages that can display data from the server, making it easy to build interactive applications.

3. Logic in Templates

With EJS, you can include control flow statements like if-else conditions, loops, and more within your templates. This allows you to handle logic directly in the views, making it easy to manage and render content conditionally.

4. Code Reusability

EJS supports the inclusion of partials, which are reusable snippets of code. This means you can break down your HTML into smaller components and reuse them across different pages, promoting DRY (Don't Repeat Yourself) principles.

5. Integration with Express

EJS integrates well with the Express framework in Node.js. It can be used as the view engine for rendering HTML pages, making it a popular choice for full-stack JavaScript applications.

6. Server-Side Rendering

EJS enables server-side rendering of web pages. This means the HTML is generated on the server and sent to the client, which can improve SEO and performance for initial page loads.

7. Flexibility

EJS templates provide a flexible way to inject data and logic into your HTML. You can pass variables from your server-side code to your templates, allowing for a high degree of customization and flexibility.

4.3.2. Node JS:

Node JS is also an open-source cross platform runtime environment compatible with multiple operating systems. Following are the advantages of using Node JS:

Efficient Development:

Node JS is commonly used for backend development. It promotes the use of JavaScript for efficient sharing of code between client-side and server-side phases. Using same language (JS) allows effective reusability of code and switching of content between different phases ensuring efficient development.

V8 Engine:

Node JS is embedded in V8 JS engines which are extremely fast. This allows different code components to be compiled in machine code directly without much effort promoting high speed and performance. Thus, node JS's overall demand increases for applications requiring high speed responses.

Data Scalability:

Node JS can handle large amounts of loaded data connections efficiently. Thus, the data scaling across multiple devices or platforms becomes easy and saves developer's time. Data scalability can be both vertical and horizontal.

OS Compatibility:

Node JS can operate on several operating systems such as windows, Linux etc. This allows efficient generation of code once and deploying across multiple platforms which save developers time, effort, and cost.

Chapter# 5

5. Testing:

Testing is a crucial phase after implementation of the software. IBM defines software testing as "Software testing is the process of evaluating and verifying that a software product or application does what it's supposed to do". In simple words testing can be defined as "The process of execution of software application with the intent catching bugs/errors and ensuring the quality, functionality and performance of the software."

Therefore, testing is done to catch bugs and enhance overall quality and performance of the software. Successfully testing can reveal a lot of bugs/errors but system is always prone to errors and cannot be 100% error free. Using different testing techniques can reduce the number of errors in the software.

5.1. Testing Technique:

The testing technique used for this software is UAT (User acceptance testing). When a specific use case or module is implemented, testing is performed to check whether the module implemented is fulfilling the desire functionality as defined in the requirements phase. UAT is performed for each use case that is defined in the requirements analysis phase.

5.2. Test Cases:

Test Case# 1:

Test Case# 1	Register
Description	This test case verifies that user has registered his/her account successfully
Set-up	User has internet connection
Instruction	1. User will click register now button. 2. User provides the registration details (username, password) and press register button. 3. The system takes the user to login screen after confirmation of details.
Expected Result	User is directed to login page after confirmation.
Verdict (pass/fail)	Pass

Test Case# 2:

Test Case# 2	Login
Description	This test case verifies that user has successfully logged in.
Set-up	User has internet connection
Instruction	1. User will login by clicking the login now button.

	2. User provides the login details (username, password) and press login button. 3. User is directed to home screen.
Expected Result	User is directed to home screen.
Verdict (pass/fail)	Pass

Test Case# 3

Test Case# 3	Validate Origin and Destination
Description	This test case verifies that user has entered valid origin and destination.
Set-up	User is logged in.
Instruction	1. User Navigates to route planning section. 2. User enters Origin by preferred way. 3. User selects Destination from preferred way. 4. System validates location.
Expected Result	No error message is displayed after input of both.
Verdict (pass/fail)	Pass

Test Case# 4

Test Case# 4	Plan Route
Description	This test case verifies that user has routes displayed on screen.
Set-up	User has provided valid origin and destination.
Instruction	1. User enters the plan route button. 2. System displays optimal route on the display.
Expected Result	optimal route is displayed on the screen to the user.
Verdict (pass/fail)	Pass

Test Case# 5

Test Case# 5	Add Stop
Description	This test case verifies that admin has added stops information to the database successfully
Set-up	Admin is logged in.
Instruction	• Admin goes to add stop option. • Admin adds a new stop with details (name, lat, long)

Expected Result	A message is displayed “Stop information is added to the database successfully”.
Verdict (pass/fail)	Pass

Test Case# 6

Test Case# 6	Update stop
Description	This test case verifies that admin has updated stops information to the database successfully
Set-up	Admin is logged in.
Instruction	• Admin goes to update stop option. • Admin searches the stop by name. • Admin updates stop information (name, lat, long)
Expected Result	A message is displayed “Stop information is updated to the database successfully”.
Verdict (pass/fail)	Pass

Test Case# 7

Test Case# 7	Delete Stop
Description	This test case verifies that admin has deleted stops information from the database successfully
Set-up	Admin is logged in.
Instruction	• Admin goes to delete stop option. • Admin deletes stop information.
Expected Result	A message is displayed “Stop (ID) is deleted from the database successfully”.
Verdict (pass/fail)	Pass

Test Case# 8

Test Case# 8	View Stop
Description	This test case verifies that admin has viewed stops information from the database successfully.
Set-up	Admin is logged in.
Instruction	• Admin goes to view stop option.

	• Stop information is displayed to the admin on display.
Expected Result	Stop information is displayed on the screen.
Verdict (pass/fail)	Pass

Test Case# 9

Test Case# 9	Add Path
Description	This test case verifies that admin has added path information to the database successfully
Set-up	Admin is logged in.
Instruction	• Admin goes to add path option. • Admin adds a new path with details (name, stops, stop number).
Expected Result	A message is displayed "path information is added to the database successfully".
Verdict (pass/fail)	Pass

Test Case# 10

Test Case# 10	Update Path

Description	This test case verifies that admin has updated path information to the database successfully
Set-up	Admin is logged in.
Instruction	• Admin goes to update path option. • Admin updates path information (name, stops, stop number).
Expected Result	A message is displayed "path is updated to the database successfully".
Verdict (pass/fail)	Pass

Test Case# 11

Test Case# 11	Delete Path
Description	This test case verifies that admin has deleted path information from the database successfully
Set-up	Admin is logged in.
Instruction	• Admin finds the path he wants to delete. • Admin deletes path (ID) information.
Expected Result	A message is displayed "Path is deleted from the database successfully".

Verdict (pass/fail)	Pass

Test Case# 12

Test Case# 12	View Path
Description	This test case verifies that admin has viewed path information from the database successfully.
Set-up	Admin is logged in.
Instruction	• Admin finds the path he wants to view. • Admin goes to view path option. • Path information is displayed to the admin on display.
Expected Result	Path information is displayed on the screen.
Verdict (pass/fail)	Pass

Test Case# 13

Test Case# 13	Manage fares
Description	This test case verifies that admin has managed fare information to the database successfully
Set-up	Admin is logged in.
Instruction	• Admin goes to route option. • Admin search route of which the fare is to be managed. • Admin manages(add/update/delete) the fare for that route.
Expected Result	A message is displayed "Fares are managed(added/updated/deleted) to the database successfully".
Verdict (pass/fail)	Pass

Test Case# 14

Test Case# 14	Add Route
Description	This test case verifies that admin has added route information to the database successfully.
Set-up	Admin is logged in.

Instruction	• Admin goes to add route option. • Admin adds route information (name, path, status).
Expected Result	A message is displayed “Route (ID) is added to the database successfully”.
Verdict (pass/fail)	Pass

Test Case# 15

Test Case# 15	Update Route
Description	This test case verifies that admin has updated route information to the database successfully.
Set-up	Admin is logged in.
Instruction	• Admin goes to update route option. • Admin updates route information (name, path, status).
Expected Result	A message is displayed “Route (ID) is updated to the database successfully”.
Verdict (pass/fail)	Pass

Test Case# 16

Test Case# 16	Delete Route

Description	This test case verifies that admin has deleted route information from the database successfully.
Set-up	Admin is logged in.
Instruction	• Admin finds the route he wants to delete • Admin goes to delete route option. • Admin deletes route information.
Expected Result	A message is displayed "Route (ID) is deleted from the database successfully".
Verdict (pass/fail)	Pass

Test Case# 17

Test Case# 17	View Route
Description	This test case verifies that admin has viewed route information from the database successfully.
Set-up	Admin is logged in.
Instruction	• Admin finds the route he wants to view • Admin clicks on view route option. • Route information is displayed on the display.

Expected Result	Route information is displayed on the screen.
Verdict (pass/fail)	Pass

Test Case# 18

Test Case# 18	Manage Route Status
Description	This test case verifies that admin has managed route status successfully.
Set-up	Admin is logged in.
Instruction	• User goes to manage route option. • User searches the route (ID) he wants to change status of. • User manages the status.
Expected Result	Route is updated with status.
Verdict (pass/fail)	Pass

Test Case# 19

Test Case# 19	Manage Profile
Description	This test case verifies that user has managed his profile successfully.

Set-up	User is logged in.
Instruction	• User goes to profiles option. • User manages his profile (username, password, themes).
Expected Result	Profile is updated on refreshing.
Verdict (pass/fail)	Pass

Test Case# 20

Test Case# 20	Report Problems
Description	This test case verifies that user has reported problems successfully.
Set-up	User is logged in.
Instruction	• User finds that some route is not working or being displayed. • User writes a report and hits send.
Expected Result	A message is displayed “report submitted”.
Verdict (pass/fail)	Pass

Test Case# 21

Test Case# 21	Logout
Description	This test case verifies that user has logged out successfully.
Set-up	User is logged in.
Instruction	• User presses the logout button. • User is directed to login page.
Expected Result	Login screen is displayed to the user.
Verdict (pass/fail)	Pass

Chapter# 6:

6. Conclusion:

The projects aim was met successfully as a user-friendly mobile application was developed providing the users with the functionalities such as:

- Registration and login.
- Validation of Location
- Planning Routes
- Providing Reports
- Managing Profiles

This system helped in following ways:

- Users from layman to business professionals were facilitated with a user-friendly environment in terms of this mobile app.
- The application helped greatly in smart journey planning and was a great initiative as there was no dedicated mobile application for local transport journey planning.
- The system was well managed and allowed users to efficiently use the app for their ease.
- Overall, the system fulfilled the urgence of effective journey planning and benefited the locals up to the hilt.

6.1. Future Improvements:

Every system designed up to today has room for improvements and so does this one. Following are some intended improvements for future:

- One potential improvement needed is accuracy of route generation. The routes generated should be more precise for better travel experience.
- Another potential improvement can be considering the real time location of transports for effective tracking.

6.2. References:

- Software Engineering, a Practitioners Approach by Rogers Pressman, 7th Edition
- Draw.io (https://app.diagrams.net/)